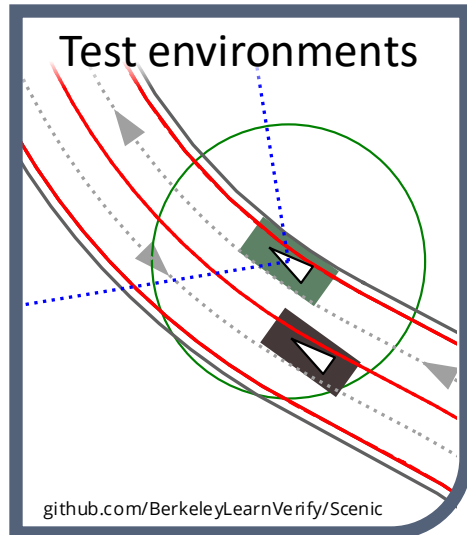
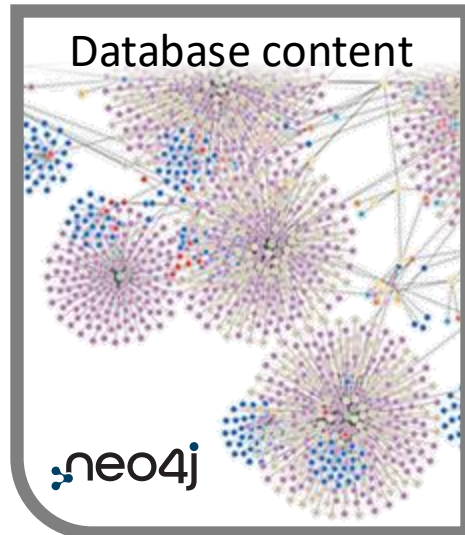
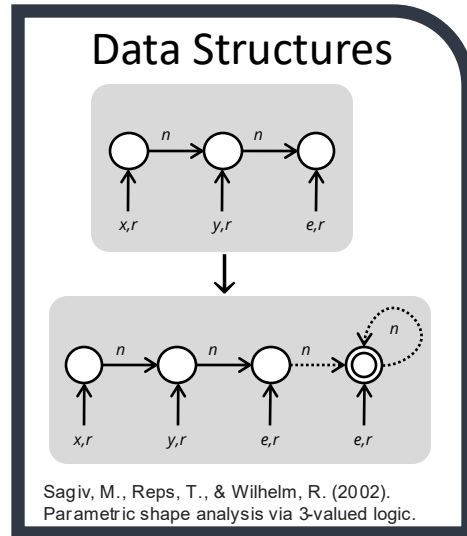
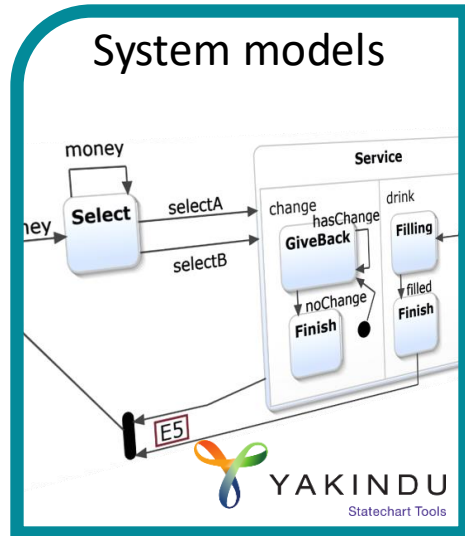


<https://refinery.tools>

Refinery: an efficient graph solver for generating well-formed models

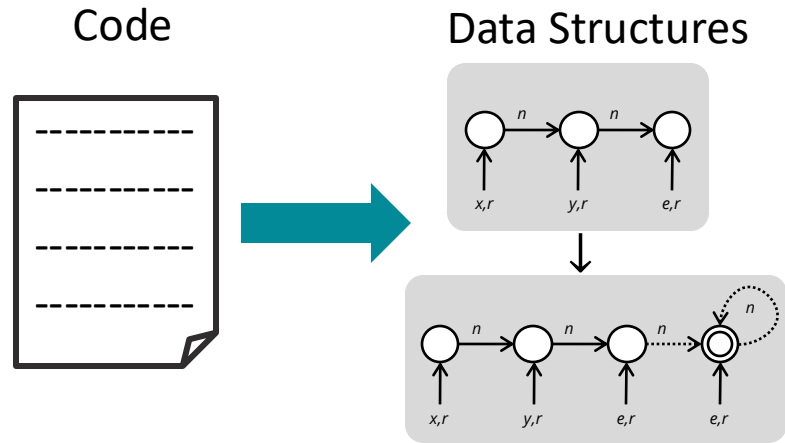


Graph-based reasoning



- Context: graph are widely used in **knowledge representation and analysis**
- Scenarios: **testing, benchmarking, or generative architecture** rely on graphs
- Approach: graph **analysis + generation** (**consistent | realistic | diverse | scalable**)
- Powerful **algorithms**
- Efficient **tools for engineers**

Initial motivation: model checking + pointers



- Model checking algorithms can be effective in verifying code with **complex behavior**
- However, existing algorithms failed to verify programs even with **simple data manipulation**
- *E.g., linked list insertion creates loop?*
- Tooling is more immature
 - Sagiv, M., Reps, T., & Wilhelm, R. (2002). Parametric shape analysis via 3-valued logic.
 - König, B. et. al. **(2024)** Counterexample-Guided Abstraction Refinement for Generalized Graph Transformation Systems

Generation overview

- How a Graph Generation framework works?



Generation overview

- How a Graph Generation framework works?

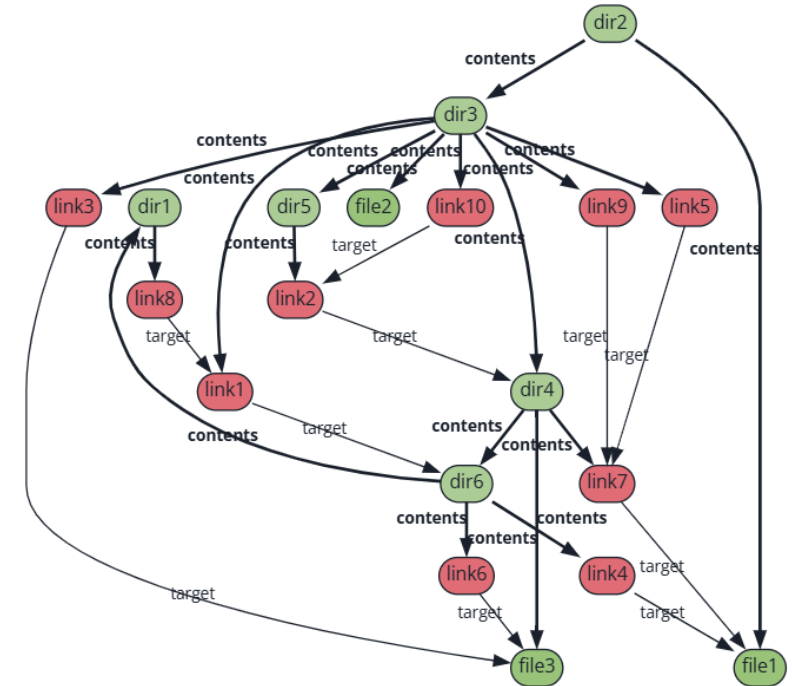
Problem

```
abstract class FSObject {  
    container Dir parent  
    opposite contents  
}  
  
class Dir extends FSObject {  
    contains FSObject[] contents  
    opposite parent  
}  
  
class File extends FSObject.  
class Link extends FSObject {  
    FSObject[1] target  
}  
  
error loopLink(l) ↔ target+(l,l).  
scope node = 20, Link = 10..*.
```

Refinery

- Size
- Number

Solution



Challenges

- Why is this so hard?

Challenge I.:
Constraints use rich logic language to characterize a wide range of relevant problems. This necessitates complex logic reasoners.

Problem



- Concepts
- Relations
- Structure
- Constraints

Refinery



- Size
- Number

Non-Trivial size $\Rightarrow$ Huge Problem

Graph



- Nodes
- Edges
- Well-formed
- Valid

Inconsistency



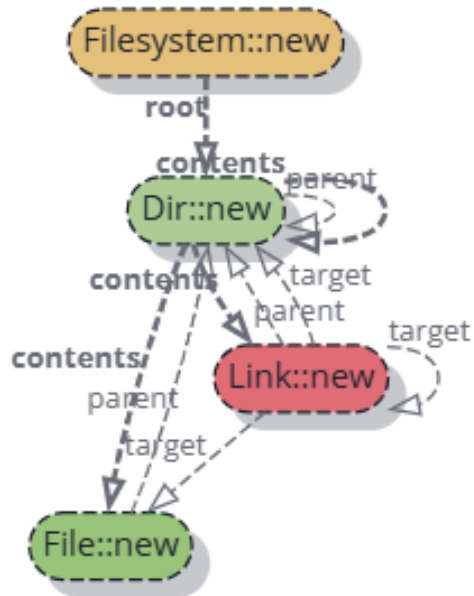
- Proof of contradiction
- Proof of completeness

Challenge II.:
The solution of the logic problem is a large, complex structure. This imposes serious performance challenge for classic solvers.

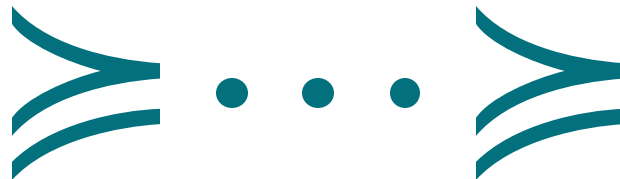
Generation overview

- How a Graph Generation framework works?

Problem

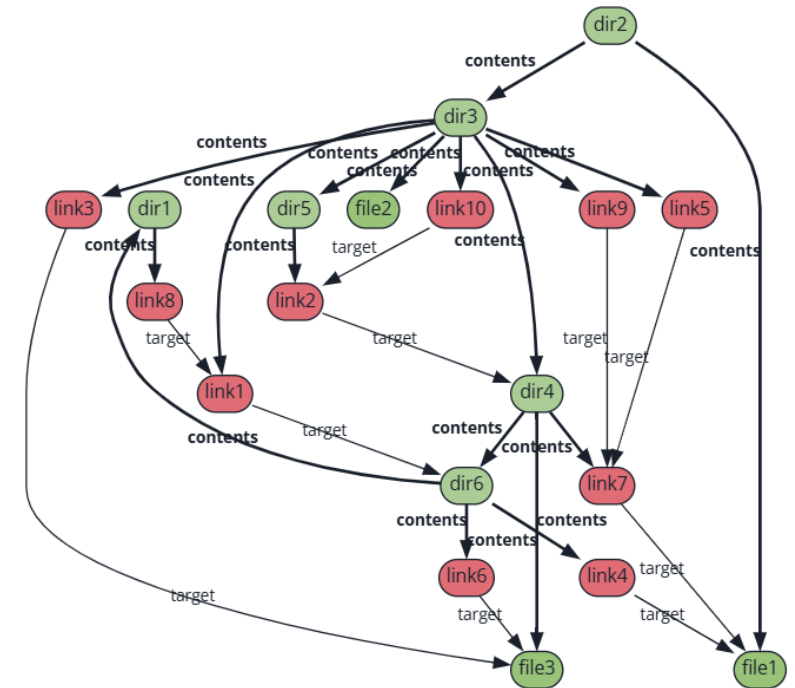


Model generation:
exploration process
that gradually reduces
uncertainty with
abstraction refinement

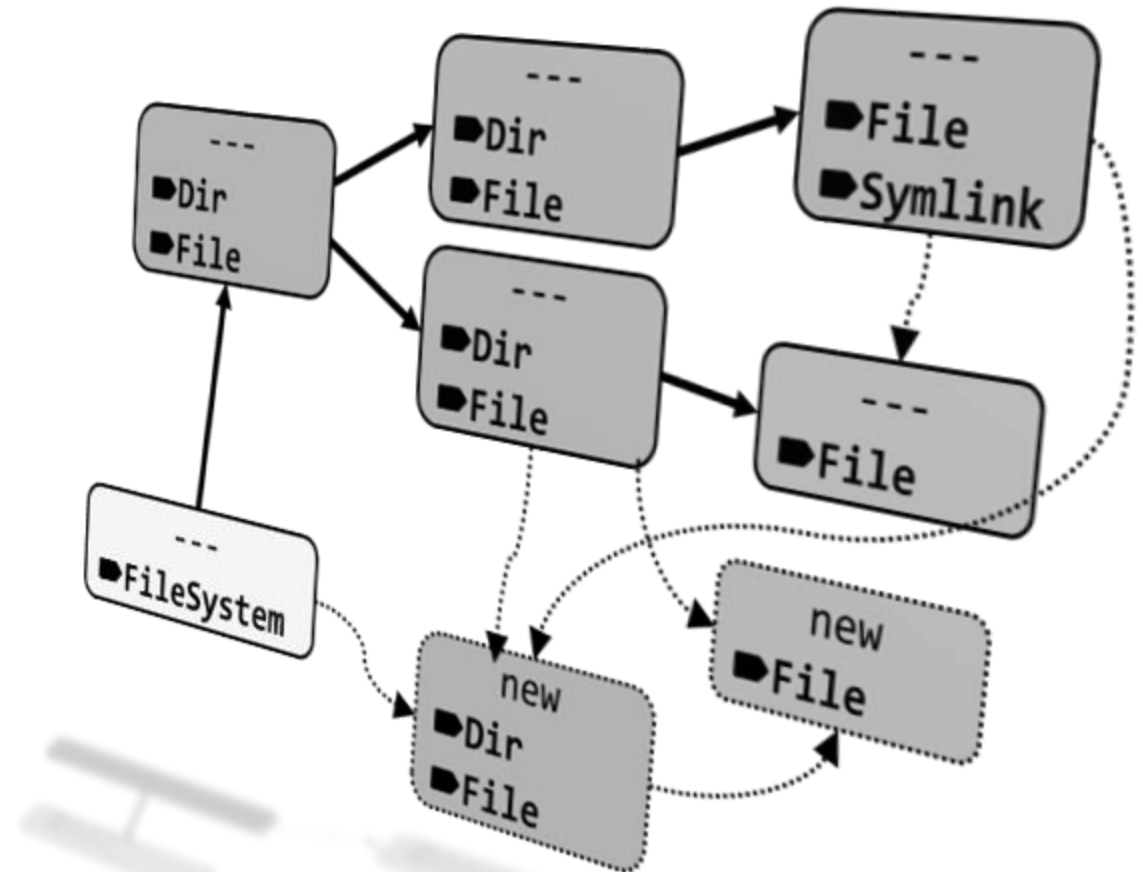


Sequence of refinement relations

Solution



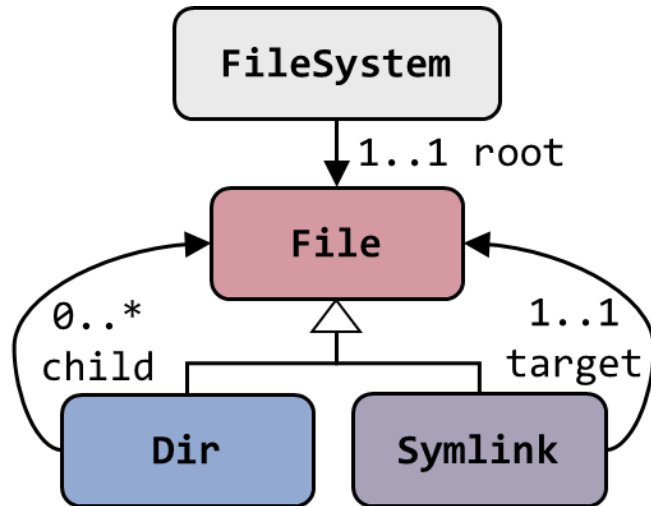
Partial Modeling



Problems and Solutions

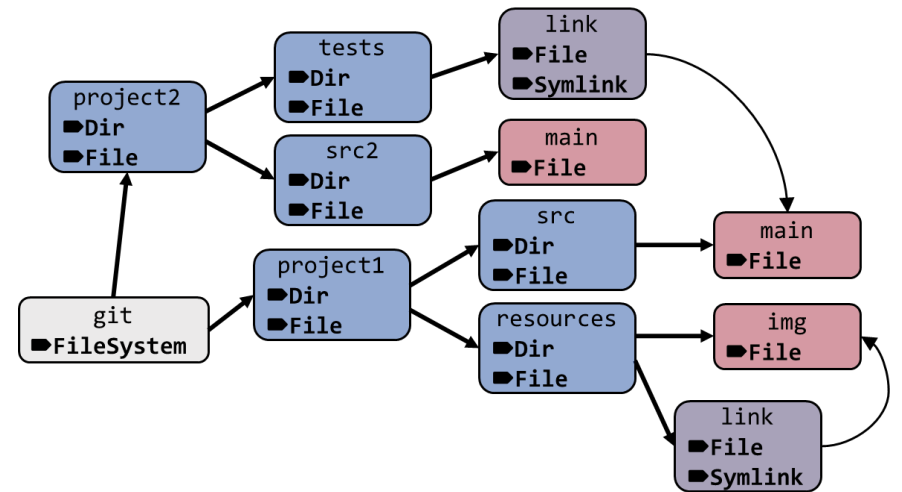
Input

Abstract models
(Metamodel + Constraints)

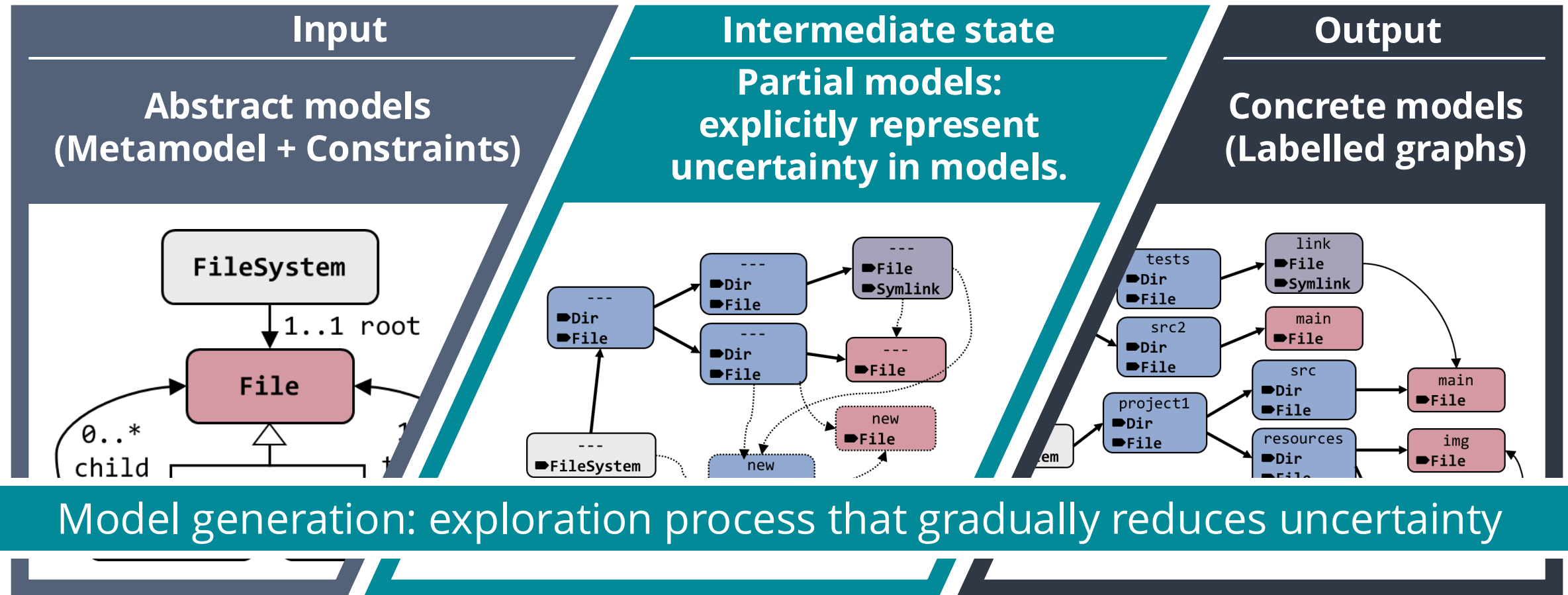


Output

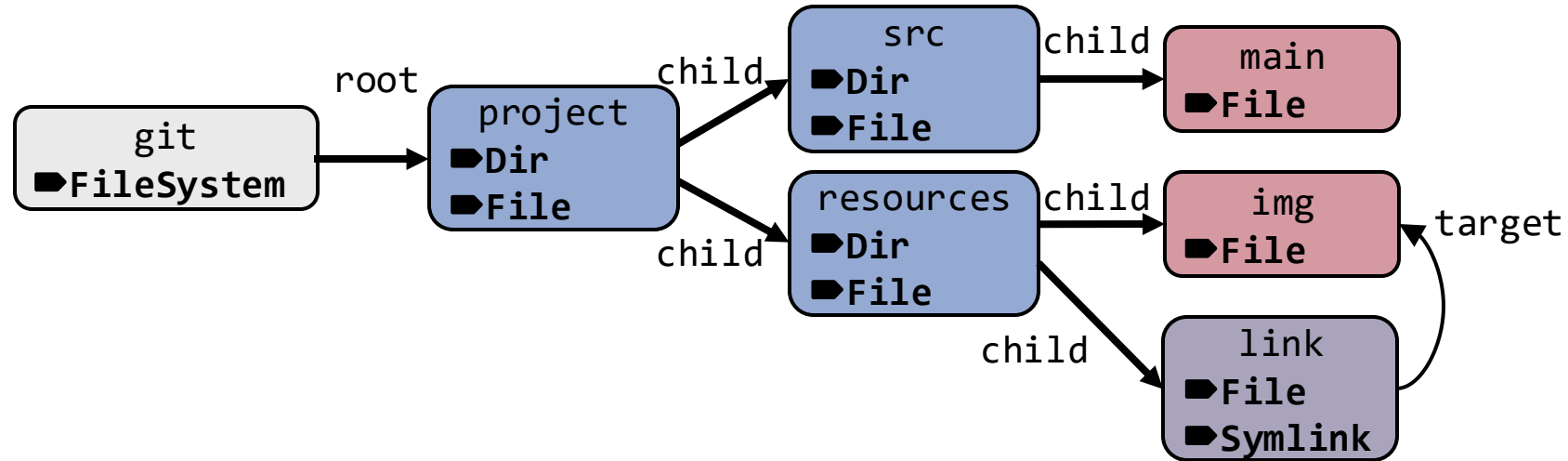
Concrete models
(Labelled graphs)



Problems and Solutions

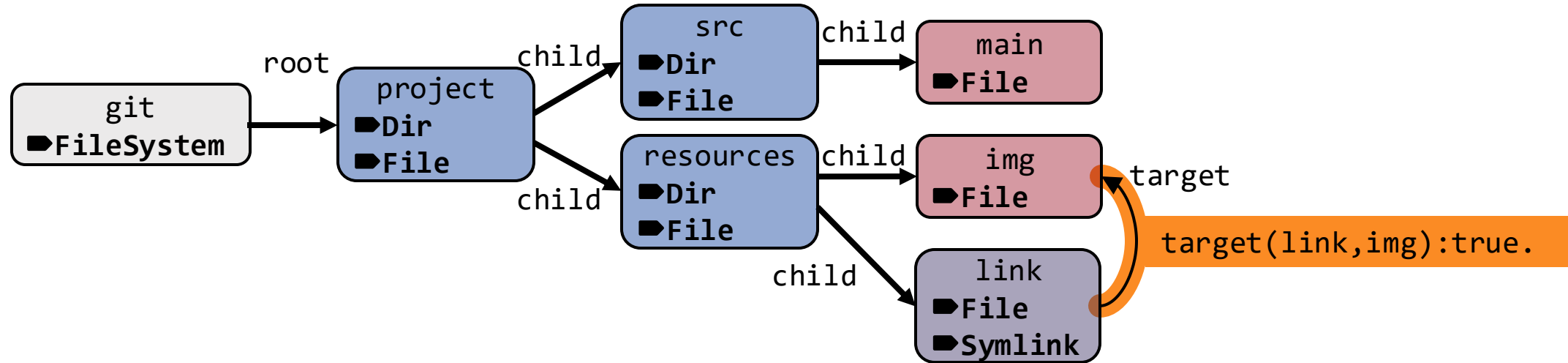


Partial modeling with 4-valued logic



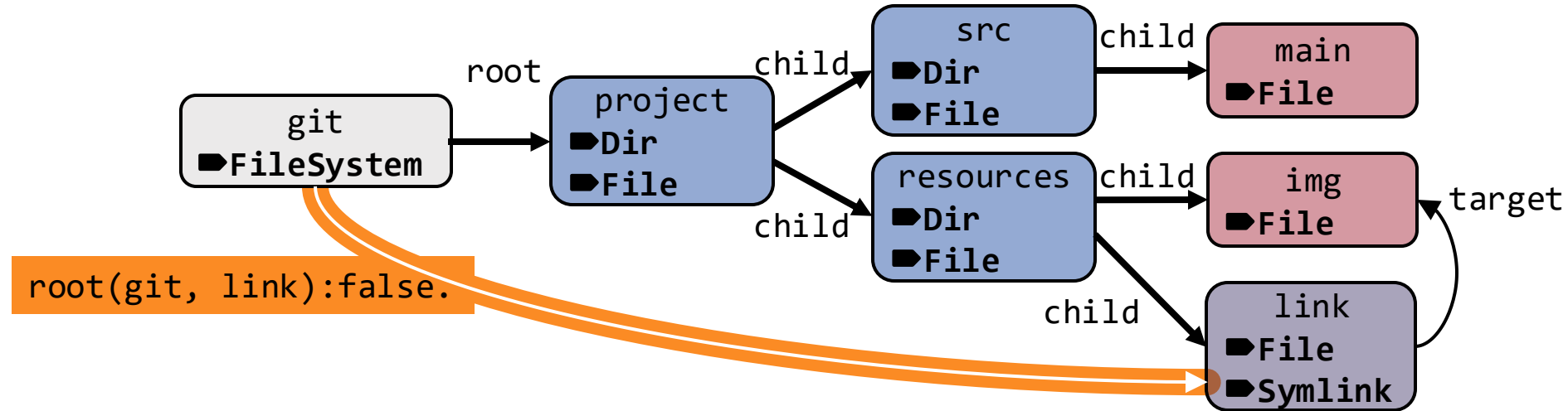
- Represent all potential extension with uncertainty
- Logic abstraction: **▣TRUE** | **False** | **▣Unknown** | **⊗Error**

Partial modeling with 4-valued logic



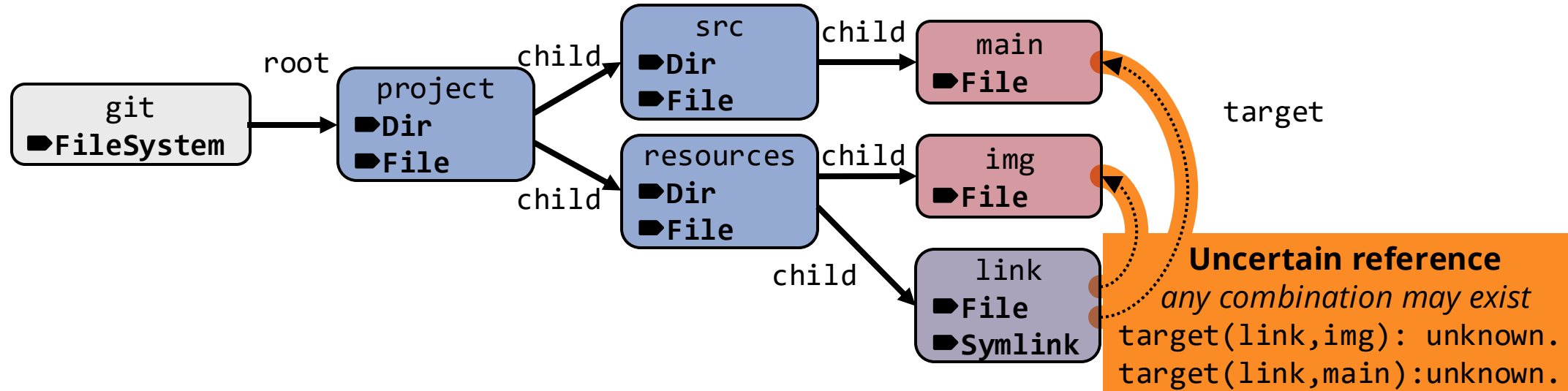
- Represent all potential extension with uncertainty
- Logic abstraction: **TRUE** | False | Unknown | ⊗Error

Partial modeling with 4-valued logic



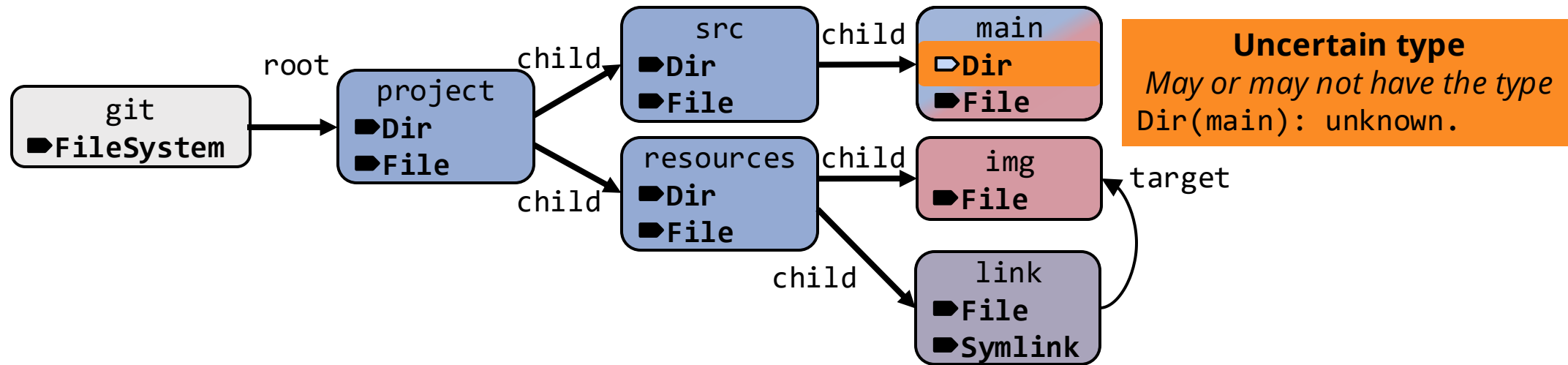
- Represent all potential extension with uncertainty
- Logic abstraction: $\blacksquare$ TRUE | **False** | $\square$ Unknown | $\otimes$ Error

Partial modeling with 4-valued logic



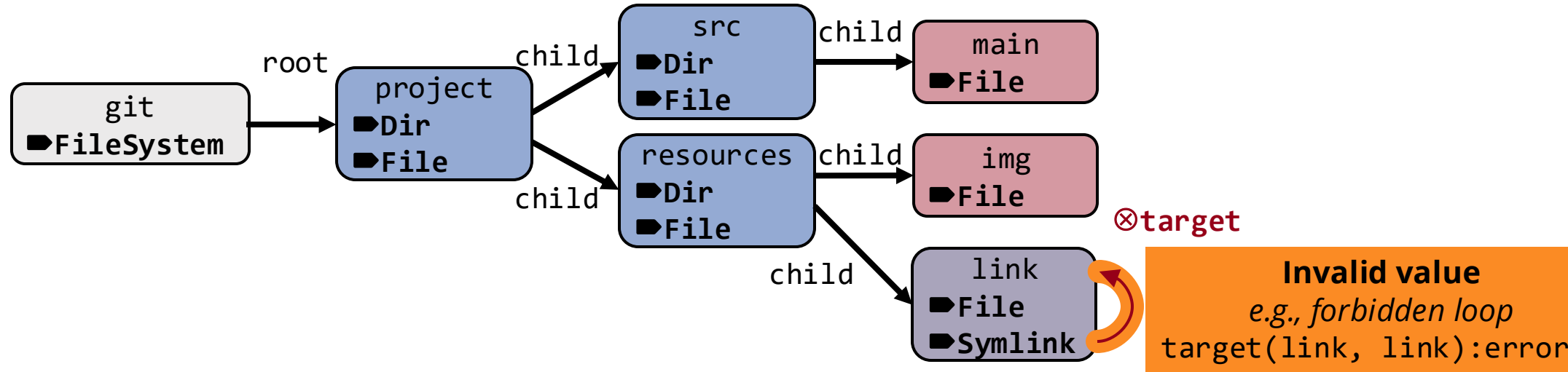
- Represent all potential extension with uncertainty
- Logic abstraction: **TRUE** | **False** | **Unknown** | **⊗Error**

Partial modeling with 4-valued logic



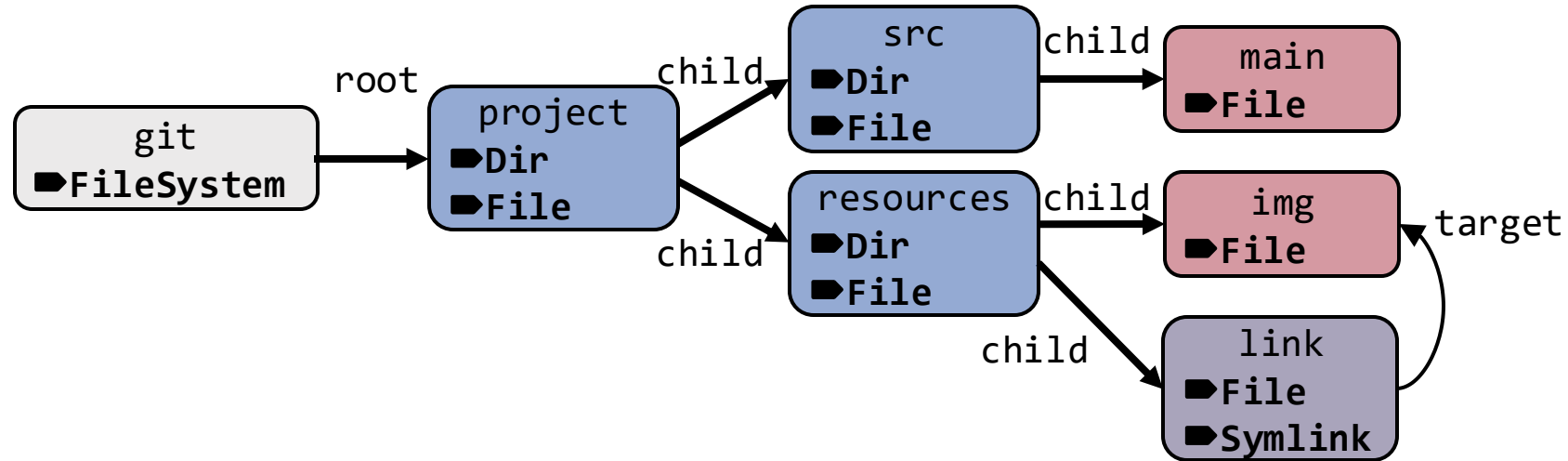
- Represent all potential extension with uncertainty
- Logic abstraction: **TRUE** | **False** | **Unknown** | **⊗Error**

Partial modeling with 4-valued logic



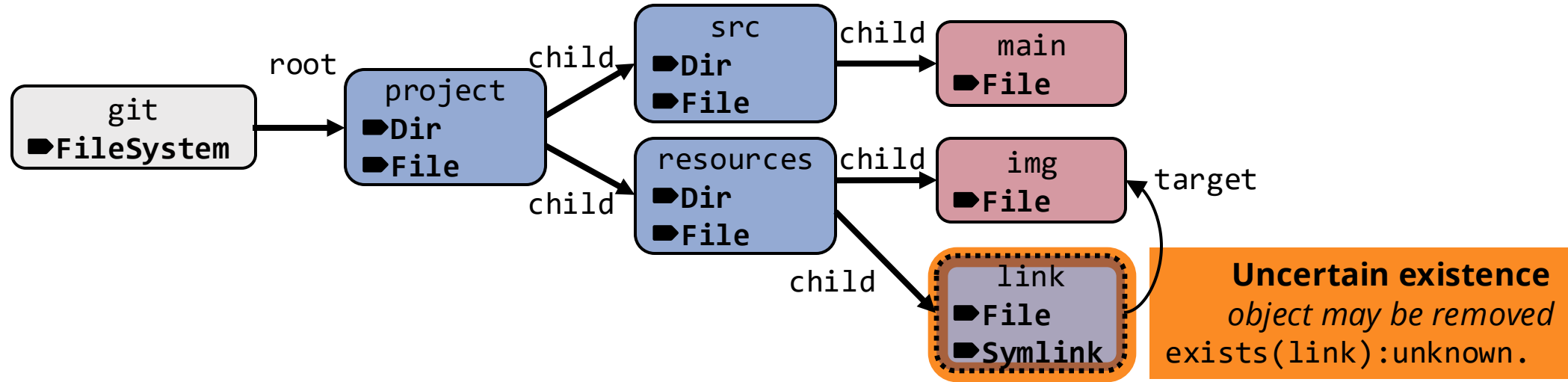
- Represent all potential extension with uncertainty
- Logic abstraction: **▣TRUE** | **False** | **▢Unknown** | **⊗Error**

Partial modeling with 4-valued logic



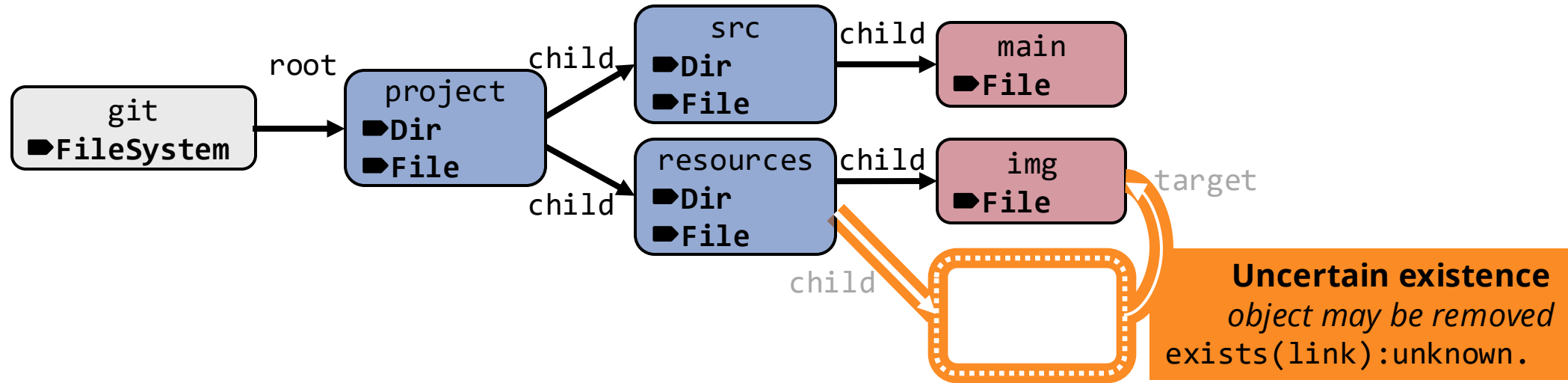
- Represent all potential extension with uncertainty
- Logic abstraction: $\blacksquare$ TRUE | False | $\square$ Unknown | $\otimes$ Error
 - 4-valued **exists**: added or removed
 - 4-valued **equals**: merging or splitting

Partial modeling with 4-valued logic



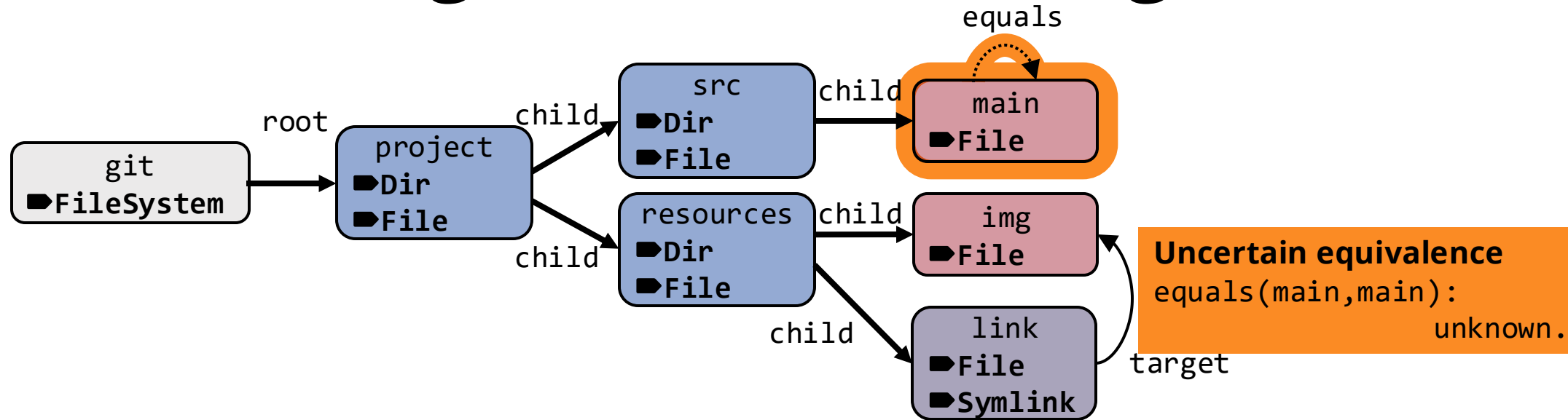
- Represent all potential extension with uncertainty
- Logic abstraction: $\blacksquare$ TRUE | False | $\square$ Unknown | $\otimes$ Error
 - 4-valued **exists**: added or removed
 - 4-valued **equals**: merging or splitting

Partial modeling with 4-valued logic



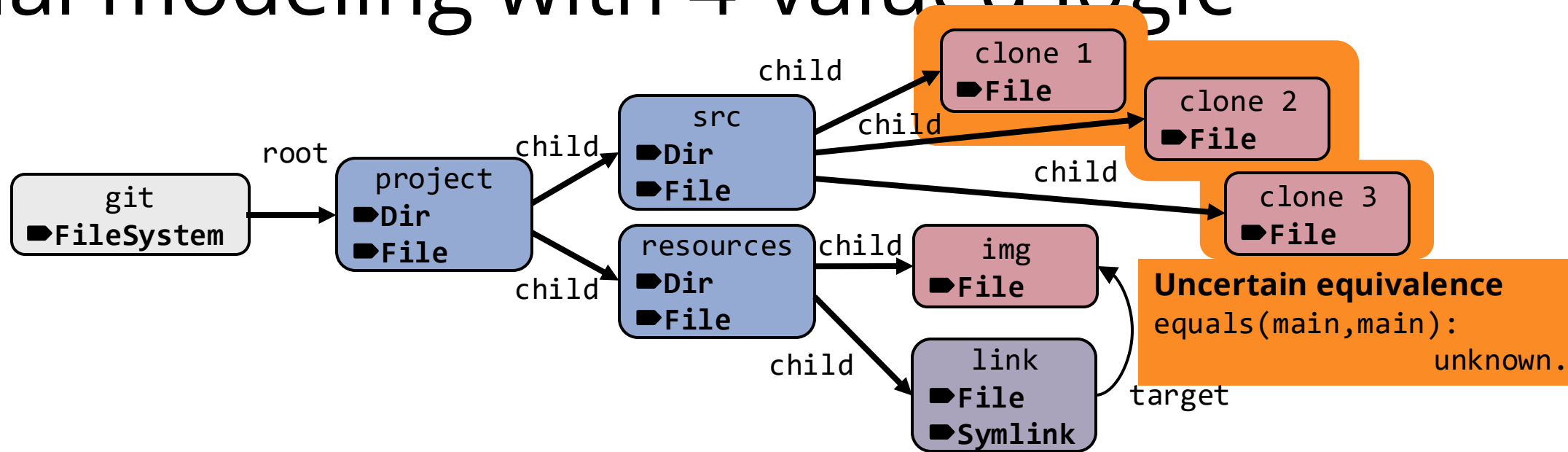
- Represent all potential extension with uncertainty
- Logic abstraction: $\blacksquare$ TRUE | False | $\square$ Unknown | $\otimes$ Error
 - 4-valued **exists**: added or removed
 - 4-valued **equals**: merging or splitting

Partial modeling with 4-valued logic



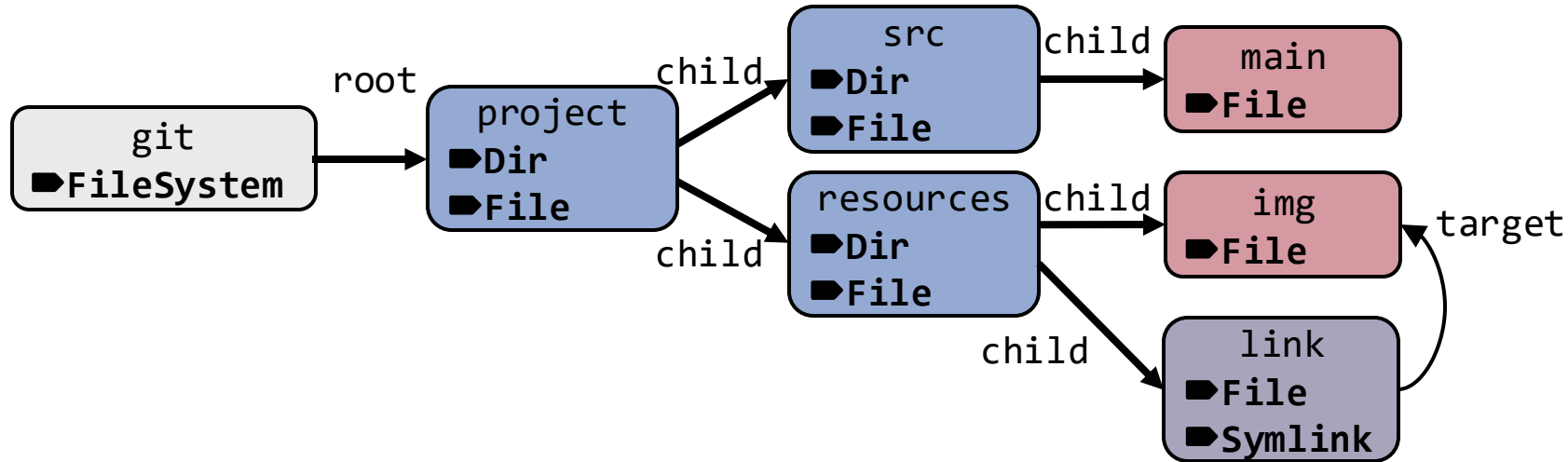
- Represent all potential extension with uncertainty
- Logic abstraction: $\blacksquare$ TRUE | False | $\square$ Unknown | $\otimes$ Error
 - 4-valued **exists**: added or removed
 - 4-valued **equals**: merging or splitting

Partial modeling with 4-valued logic



- Represent all potential extension with uncertainty
- Logic abstraction: $\blacksquare$ TRUE | False | $\square$ Unknown | $\otimes$ Error
 - 4-valued **exists**: added or removed
 - 4-valued **equals**: merging or splitting

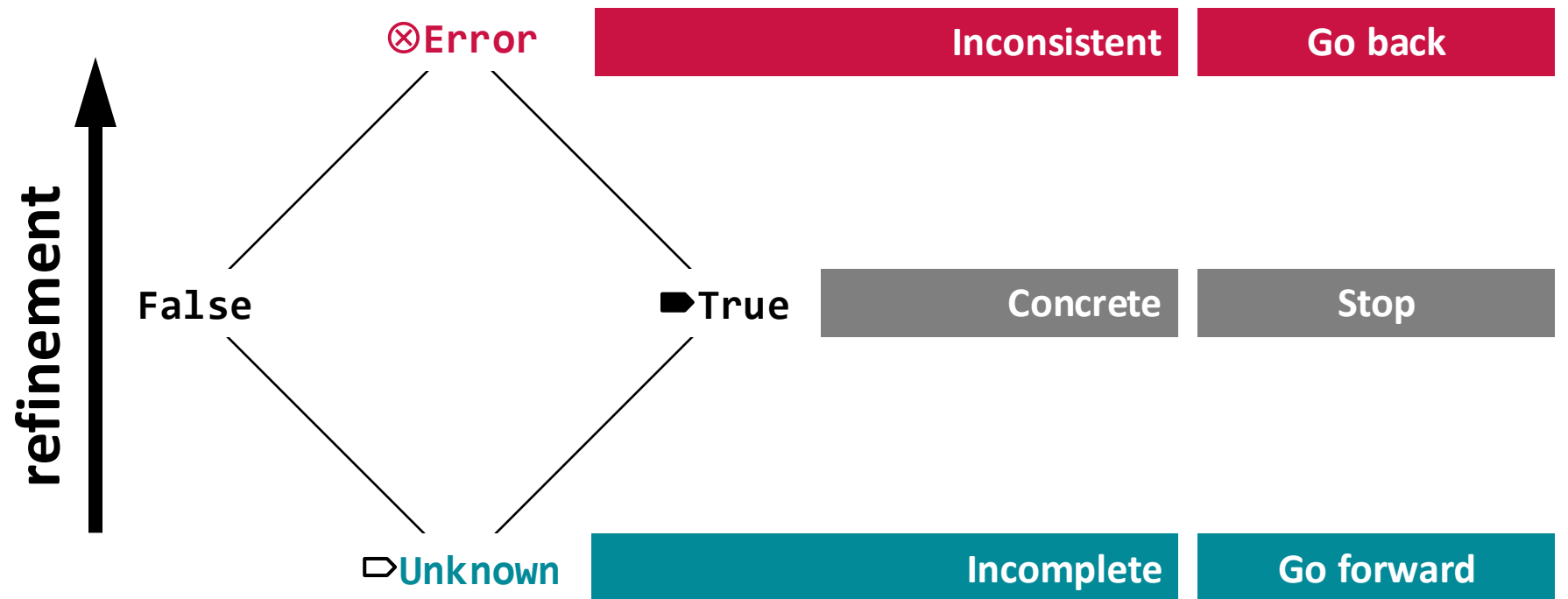
Partial modeling with 4-valued logic



- Represent all potential extension with uncertainty
- Logic abstraction: $\blacksquare$ TRUE | False | $\square$ Unknown | $\otimes$ Error
 - 4-valued **exists**: added or removed
 - 4-valued **equals**: merging or splitting
- **Refinement**: reduces uncertainty $\rightarrow$ concrete models

Refinement: 4-valued logic

- Model generation is executed with respect to model refinement

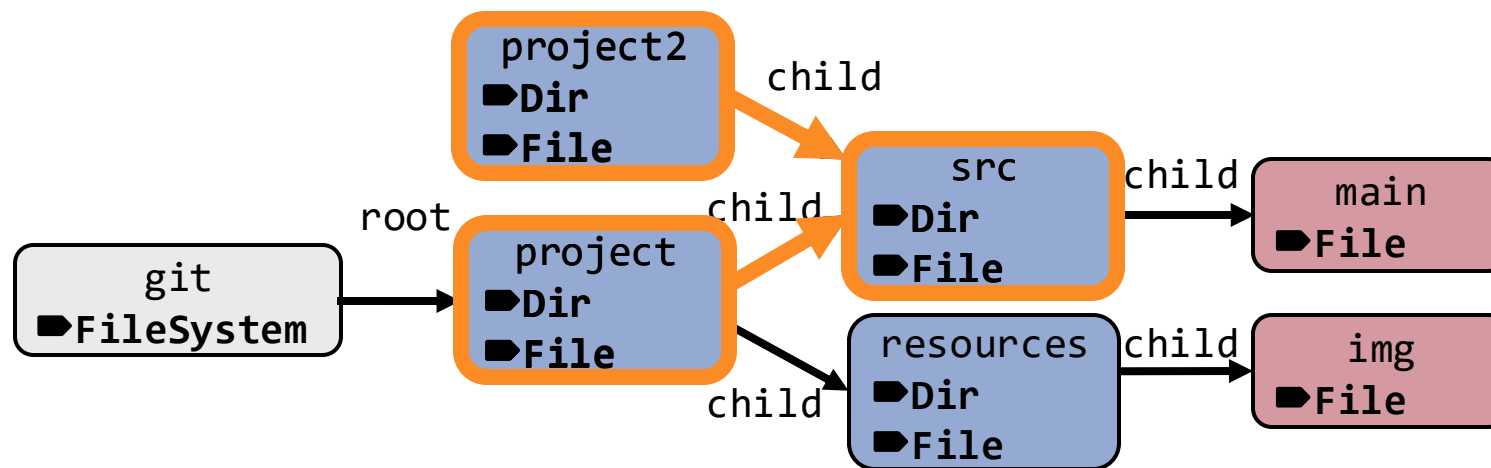


Predicate evaluation

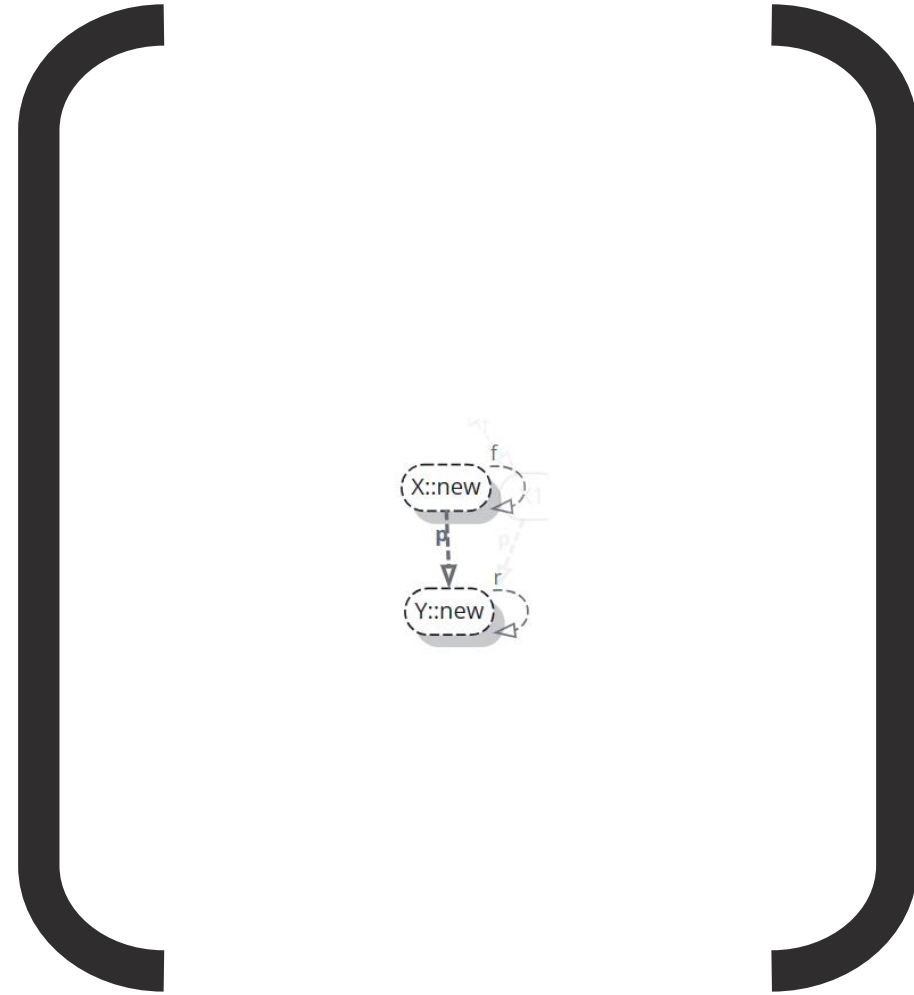
- First-order logic predicates can be defined on partial models

```
error pred twoParents(f)  $\leftrightarrow$   
  child(p1,f), child(p2,f), p1  $\neq$  p2.
```

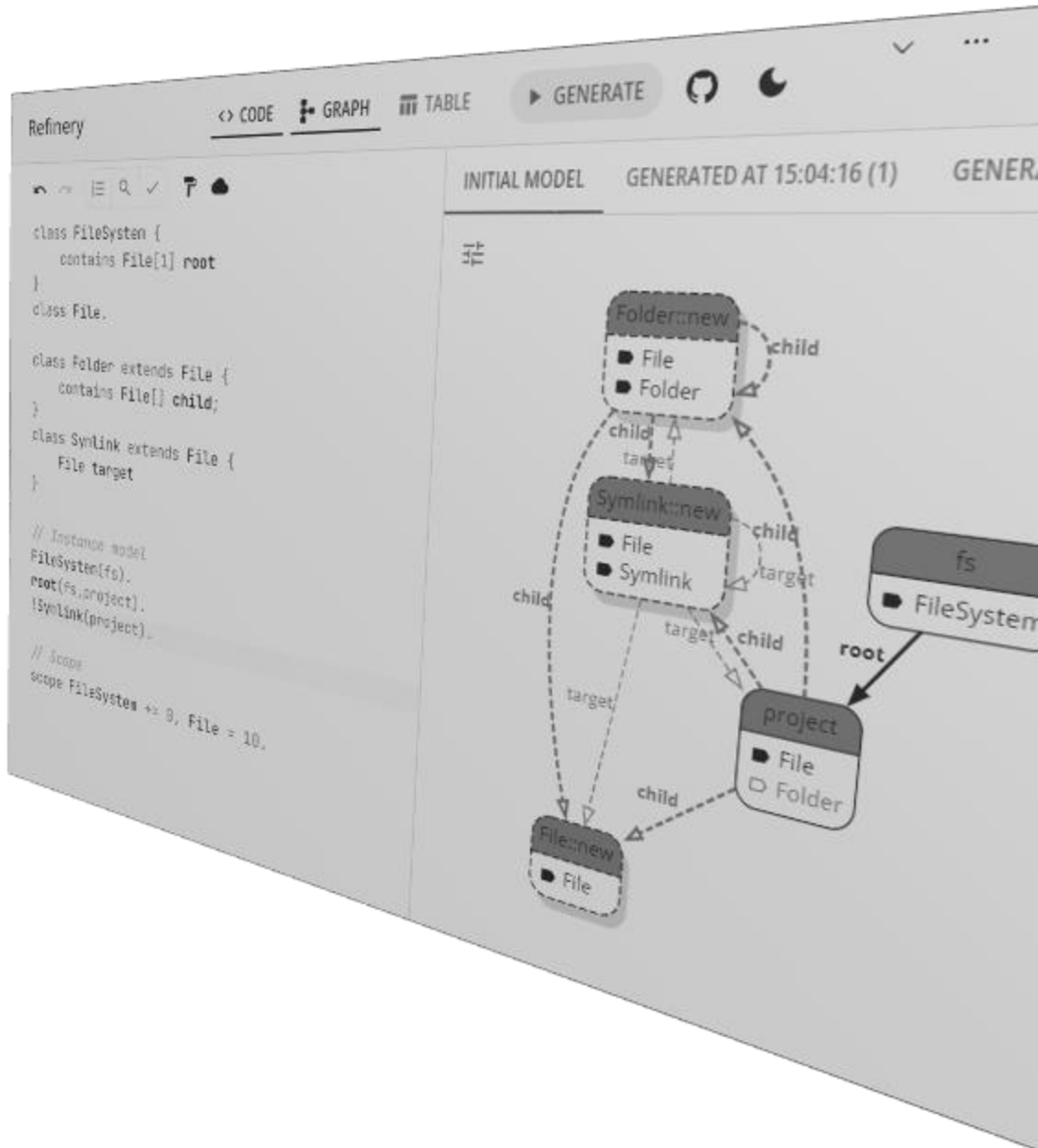
- Predicates respect refinement
- Efficiently evaluated by high-performance query engine



Exploration: automated refinement

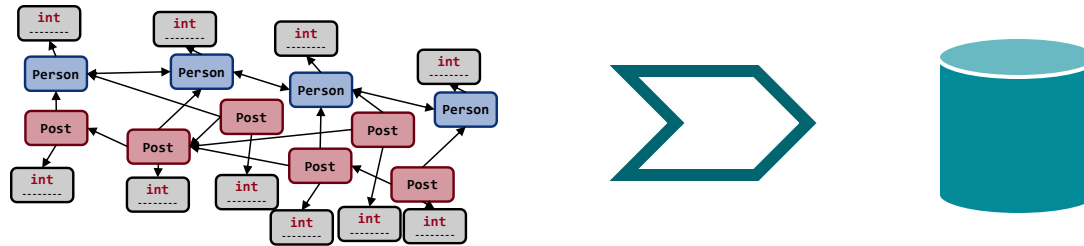


Technical background

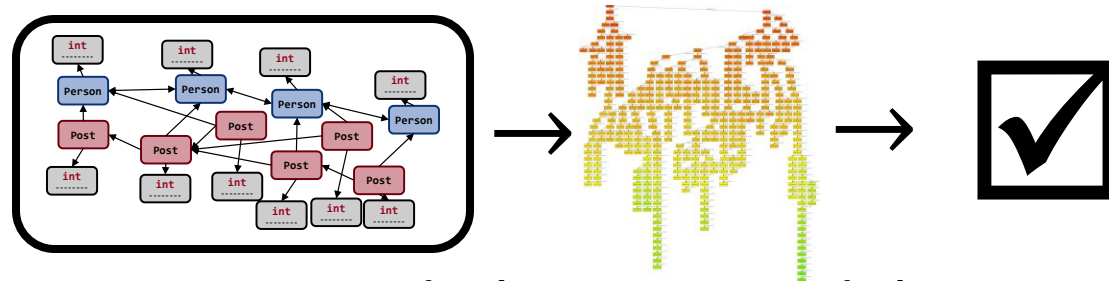


Libraries

- **Management:** *store/compare multiple versions of (abstract) models*

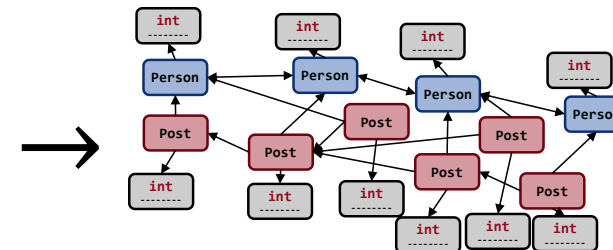


- **Exploration:** *transformation and optimization of models*



- **Reasoning:** *logic reasoning and abstract model concretization*

`crossSystemLink(p1,p2)↔`
`target(p2,p1)∧target(p1,t1)`
`∧ created(p2,t2)∧ (t2≠t1)`



Logic solver

- Constraints are continuously reevaluated
- Automatically searching for valid models by applying refinements
- Search is parametrized
 - Number of different solutions
 - Difference between the solutions (non-isomorphic)
 - Random seed
- Scope: *"size of the models"*

```
scope node = 40..50, FileSystem = 2, SymLink += 10, importantFile = 1..*.
```

of nodes

nodes by type

of new objects

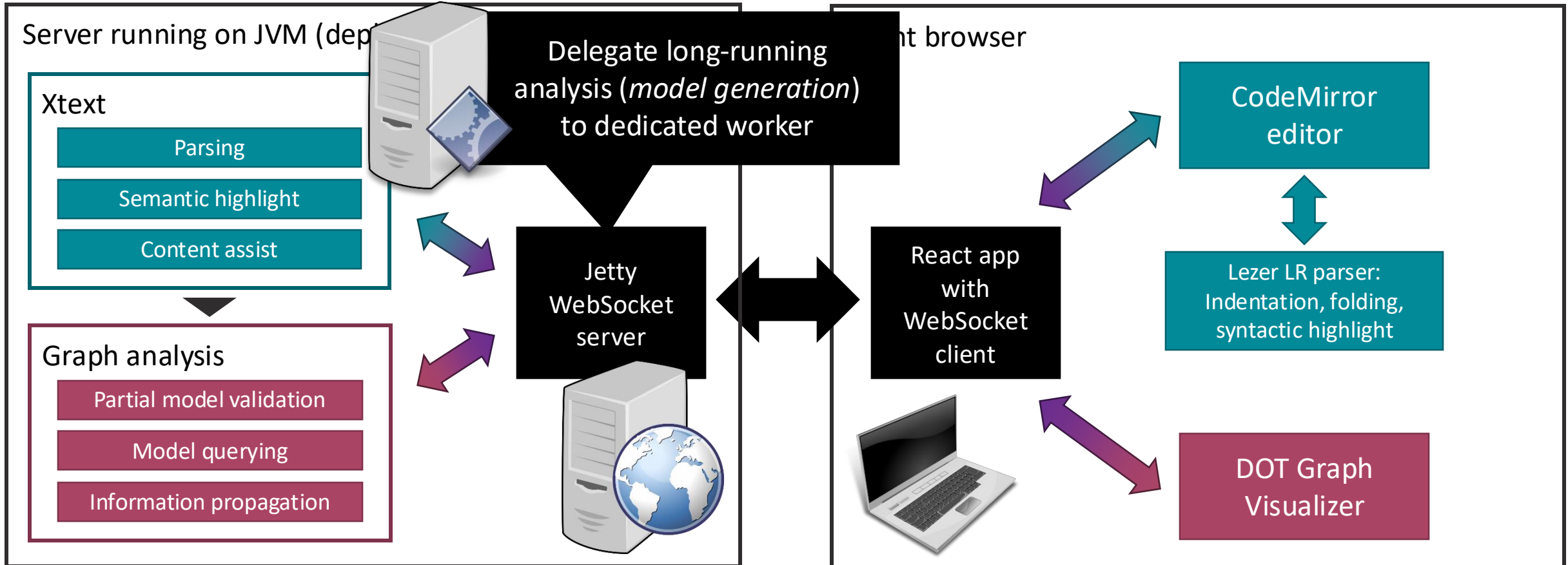
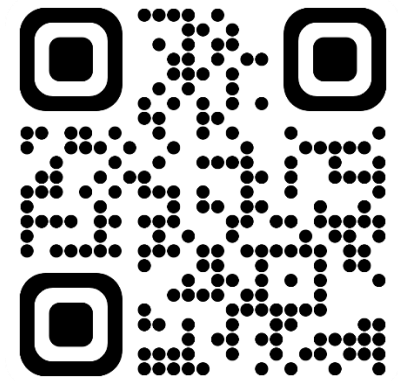
of predicate matches

	Problem size			Largest model
	#class	#ref	#error	#node
FileSys	4	2	0	19750
FAM	9	13	4	15750
Yakindu	11	7	8	4250
Ecore	20	33	1	2000
Social	2	4	2	230

Table 1: Largest models generated in 60 seconds

Online demo

- **Real-time feedback** about partial model
- **Logic reasoning** and **Model Generation** in the cloud

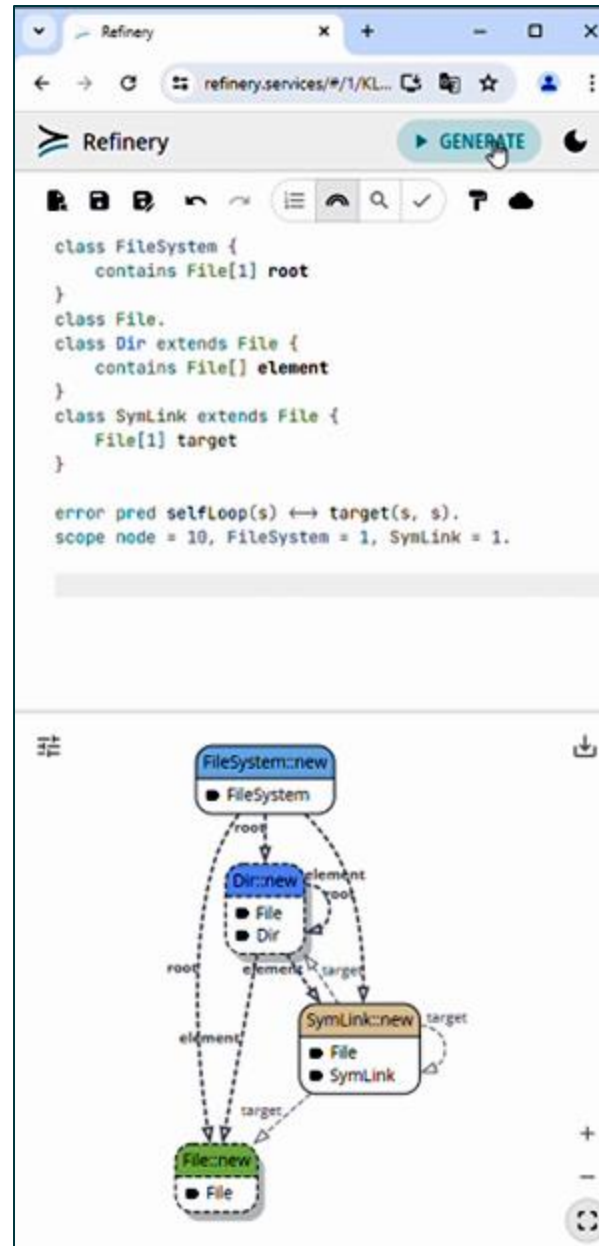


Ongoing projects



Modern tools

- Outperforms Z3 (Microsoft) and Alloy (MIT) by **100x**
- Interactive state-of-the-art **web-based editors**
- Generation algorithms **deployed on server**
- Various domains, e.g.,
*blockchain architecture | chemical reaction
railway topology | modeling environment,
satellite network | video game maps*
- **Goal:** support **engineers** and **experts** solving complex problems



Recent Results

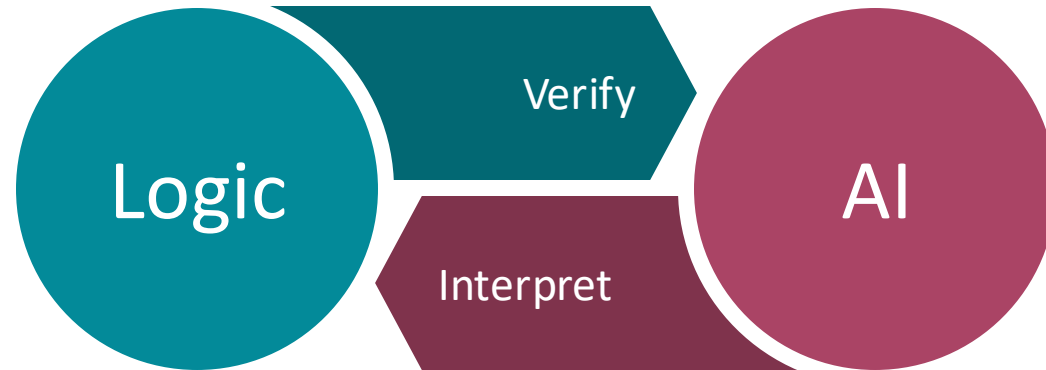
-  Refinery
Continuously deployed:
<https://refinery.tools/>

- Amazon
Research Award
First in the region

amazon | science

Logic + AI = Neuro-symbolic approach

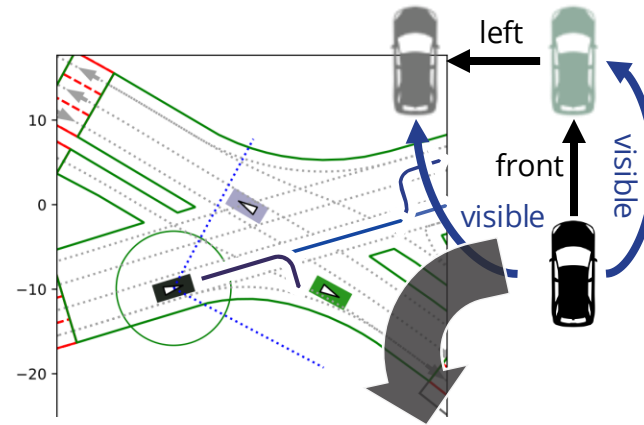
- Formal methods + Logic solvers
- With AI applications



Testing of AI/ML Applications



- AI applications are **data-oriented** systems
- **Complex, dynamic** environment
- Novel **generation** + Advanced **simulators**
→ Diverse **tests**
- Systematic testing of AI applications



Recent Results

Map generation:
Knorr Bremse
Prolan

Research project
with USA Navy

Office of Naval Research Global



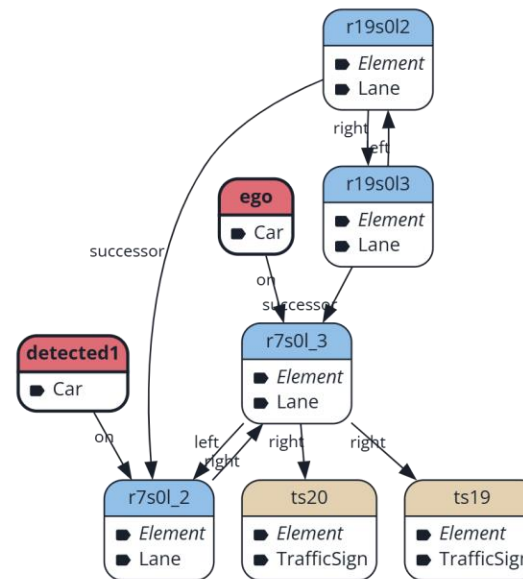
AI output analysis

- Process the output of unreliable AI components
- Synthesize explanation + improvements

+Integrity
+Reliability



analysis   synthesis



- Research project with the European Space Agency (IncQuery Labs + Integricom)
VERIFIABLE AI/ML TECHNIQUES FOR PNT APPLICATIONS



- LLM + Logic reasoning
Research collaboration with industrial partners

► Demo: ESA-VAMPIR

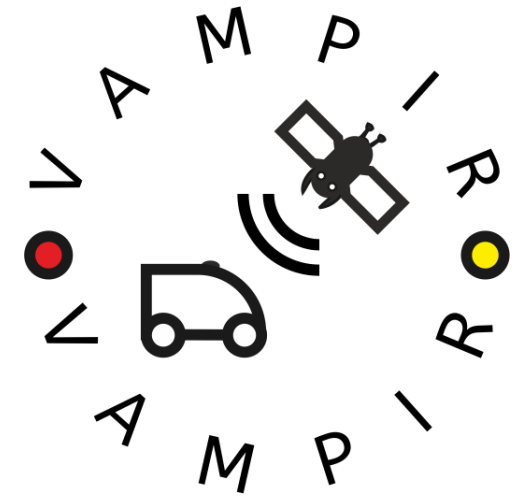
1. Input:

- GNSS position + HD-Map
- AI sensor output

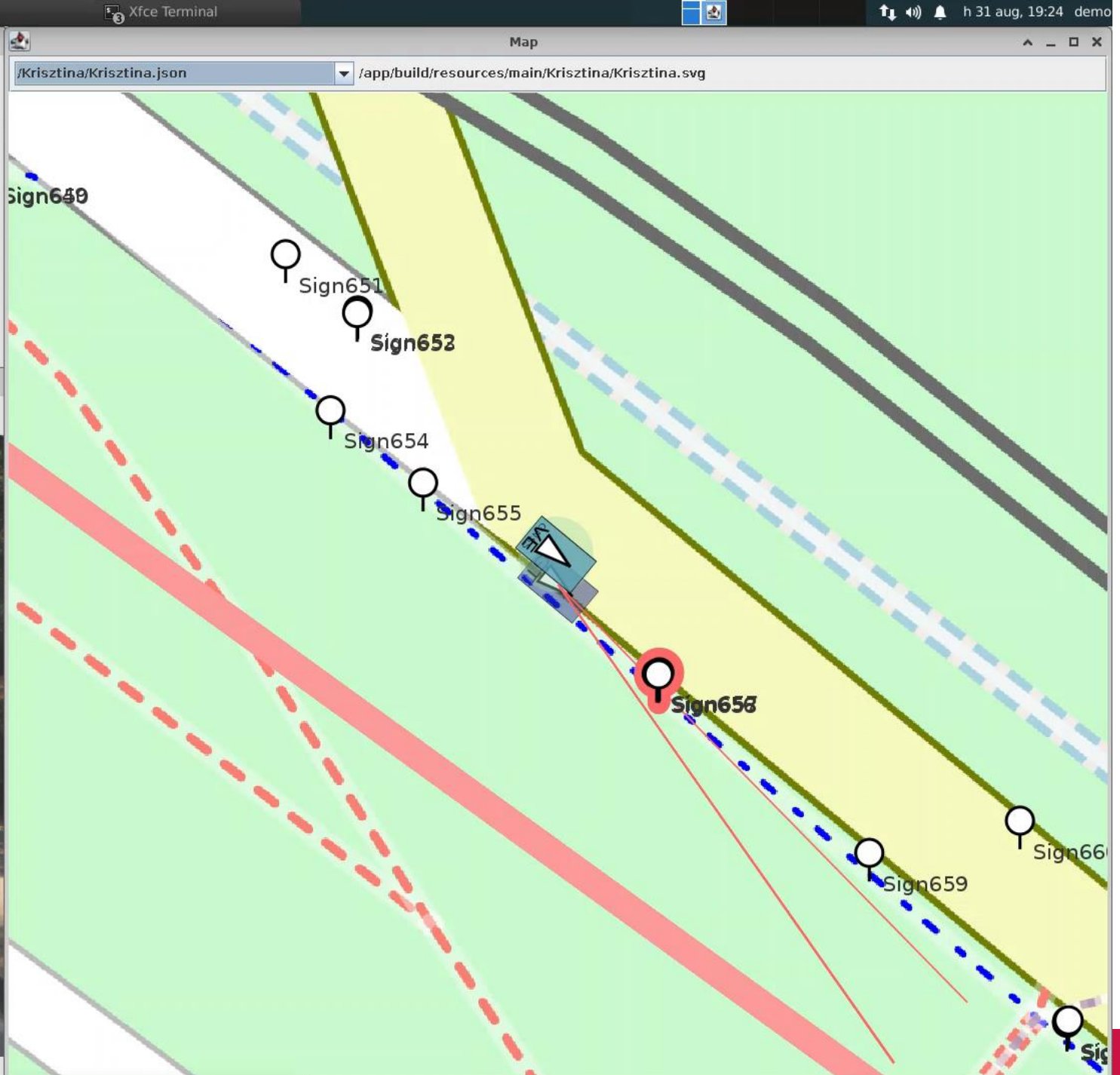
2. Logic reasoning with Refinery

3. Output:

- Level 1 > Scene interpretation
- Level 2 > Safety and accuracy
- Level 3 > Improved GNSS accuracy



Experimental Evaluation							
					Verification Engine		
					USE		DoNotUse
					98%		2%
					VALID	MISLEADING	
					54%	46%	
GNSS	USE	100%	VALID	28%	19	0	1
			MISLEADING	72%	18	31	1
	DoNotUse	0%			0	0	0



► Demo: AI-assisted modeling with precision

The screenshot shows the Refinery application interface. The top bar includes the 'Refinery' logo, navigation icons, a lock icon, a 'GENERATE' button, and a settings icon. The main workspace is divided into two panels. The left panel, titled 'INITIAL MODEL', shows a state machine diagram with a single state and a self-loop. The right panel displays the AI-generated output, which is a state chart for a traffic light. A teal callout box labeled 'Prompt' points to the text 'create me a state chart of a traffic light.' in the right panel. At the bottom right, there is a text input field with the placeholder 'Type a message' and a send button.

Refinery <> [Icons] [Lock] GENERATE [Settings]

INITIAL MODEL GENERATED AT 07:42:06 (0) [Close]

[Diagram Icon] [Download Icon]

create me a state chart of a traffic light.

Prompt

Type a message [Send]

► Demo: AI-assisted modeling with precision

Refinery <>

INITIAL MODEL

GENERATED AT 07:42:06 (0)

example, if there were a specific starting sequence or initial state handling beyond the first transition. The 'noEntryInRegion' constraint will therefore be violated in the initial setup, which I've addressed by commenting on its absence.

AI response contains semantic errors

The error `noEntryInRegion(r1)` indicates that the `Region r1` does not contain any `Entry` pseudostate. To fix this, I will add an `Entry` pseudostate to `r1` and then add a transition from this `Entry` to the `redState` to establish the initial state of the traffic light.

Successfully generated model

Type a message

Solution

Prompt

Validation + Iteration

Accept

Summary



Summary of Refinery

- **Logical reasoning** and **model generation** over graphs
- **Web-based editor:**
 - **Live editing** and **feedback**
 - Support for partial models and graph constraints
- **Containerized execution:**
 - Continuously deployed at <https://refinery.services/>
 - Available as **Docker image**: <https://refinery.tools/learn/docker/>
- **Framework** for graph processing tasks

